

llama.cpp Throughput for Qwen3.8-Flash-Next and gpt-oss-120b on a 62 GiB RAM Laptop

Matthew Schwartz

Independent researcher [ORCID 0009-0009-4171-7247](https://orcid.org/0009-0009-4171-7247) matthew.schwartz.research@gmail.com

September 2026

Abstract

Large mixture-of-experts (MoE) models can be deployed across system memory, GPU memory, and storage, but file size alone does not predict generation speed. We measured two public GGUF deployments with unmodified llama.cpp on an Intel Core i9-14900HX laptop with 62 GiB of RAM and an 80 W RTX 4080 Laptop GPU. The engine used its ordinary memory-mapped file path, with routed experts assigned to the CPU and GPU offload requested for other supported layers. The 67.564 GiB Qwen3.8-Flash-Next artifact decoded at 9.4–14.7 tokens/s across two sequential three-run groups under different recorded host states. A 59.034 GiB gpt-oss-120b artifact decoded at 8.7–9.3 tokens/s across three runs. Each invocation requested 256 output tokens from one fixed prompt. These are decode rates: they exclude model loading and prompt processing and do not establish interactive response latency. The report provides run-level measurements, exact deployment identities, and separate small task checks as a reference for reproducing these configurations. Sequential run groups and whole-device storage counters leave the causes of the observed rates unresolved. The contribution is a hardware-specific throughput benchmark; no inference method was changed.

1 Introduction

A local deployment must deliver answers within a useful time budget as well as fit the available memory. With 59–68 GiB model files on a 62 GiB RAM laptop, the operating system, inference buffers, and weights compete for system memory, while a 12 GB GPU holds only part of the deployment. Measuring generation speed is one step toward deciding whether such a configuration can serve the intended workload.

Linux memory mapping lets an engine address file-backed pages without first copying the whole artifact into RAM [1, 2]. llama.cpp users have already reported large, storage-backed MoE deployments through this path [3, 4]. The open practical question for a new configuration is its measured speed, rather than whether memory mapping permits execution at all.

Mixture-of-experts models make this performance question especially relevant. They store many feed-forward experts but route each token through only a few [5–7], so stored size and active computation are not the same quantity. Purpose-built runtimes exploit this structure through expert placement, prefetching, compression, and caching [8–13]. The ordinary mmap path in llama.cpp provides a reference configuration for evaluating these techniques [14].

We ask how fast Qwen3.8-Flash-Next and gpt-oss-120b decoded on this laptop with a fixed llama.cpp checkout and CPU/GPU placement. “Unmodified” refers to the source checkout; the command lines use explicit placement flags.

The 67.564 GiB Flash deployment completed generation at 9.4–14.7 decode tokens/s across its recorded host states, and the 59.034 GiB gpt-oss deployment ran at 8.7–9.3 tokens/s. The contri-

bution is this benchmark and the configuration needed to repeat it. Tensor inventories document the files, and small separate task samples provide limited checks of the deployed models’ answers. Prompt processing is reported separately because decode speed alone does not determine response latency.

The larger file exceeded installed system RAM, but not combined nominal RAM and VRAM; the study did not measure simultaneous residency of every tensor byte. The measurements therefore do not establish execution beyond combined physical memory or identify a paging mechanism. Section 2 defines the exact deployments and measurements, Section 3 reports their observed rates and supporting audits, and the remaining sections explain how the baseline can and cannot be used.

2 Methods

2.1 Machine, engine, and placement

The host was a Linux laptop with an Intel Core i9-14900HX, 62 GiB of dual-channel DDR5 memory, a 2 TB WD PC SN740 NVMe drive, and an NVIDIA RTX 4080 Laptop GPU with 12282 MiB of VRAM at an 80 W power limit. The recorded baseline inventory lists Linux 7.0.0-29-generic, NVIDIA driver 580.167.08, and driver-reported CUDA compatibility 13.0. It also records 8 GiB of swap, with 7.3 GiB in use at that snapshot. These are inventory values, not per-run measurements of software state, memory pressure, or swap traffic. Run logs record llama.cpp build b1-035e227; the associated clean source checkout currently resolves to 035e22731a7fd70b9854b3a2d64ec68e9b1a45d3. It was compiled with CUDA for compute capability 8.9. The surviving build record identifies GNU C++ 13.3.0 and CUDA toolkit compiler 12.6.85; the release artifact records the remaining toolchain and CMake options. For the Flash runs, that revision was a commit in the then-unmerged Qwen4Exp support pull request; the pull request subsequently merged on August 27, 2026 [15]. The gpt-oss path did not depend on that experimental architecture support.

Both command lines used llama.cpp’s default mmap behavior, requested GPU offload with `-ngl 99`, and assigned routed MoE tensors to the CPU path with `--n-cpu-moe 999`. Thus the requested placement combines CPU expert execution with GPU execution of other supported layers. The exact flags and engine revision define the baseline.

2.2 Artifacts and tensor inventory

GGUF is the model container used by llama.cpp. The first subject was the three-shard Unsloth Qwen3.8-Flash-Next-UD-IQ1_S artifact at Hugging Face revision d3bc75ee6cce3efc1e228ec00a6cc2cdb1e2249. Its GGUF metadata declares 48 blocks, 512 experts, and 10 active experts per token [16, 17]. The second was gpt-oss-120b-MXFP4.gguf at revision 238abdd290bb874b90a5da1b4549881b7d05c091, with 36 blocks, 128 experts, and 4 active per token [18, 19].

We parsed the frozen GGUF headers and summed stored bytes and logical tensor elements by standardized tensor name. Effective whole-file bits per enumerated logical tensor element (bpw) is $8 \times$ artifact bytes divided by enumerated logical tensor elements. The inventory is descriptive of the deployed files; it does not measure which pages the engine read during a token.

Artifact	File GiB	File bpw	Routed experts GiB	Experts/active	Notable other tensor
Flash UD-IQ1_S	67.564	3.28	37.109	512/10	26.822 GiB lookup table
gpt-oss MXFP4	59.034	4.341	56.879	128/4	—

Table 1: Deployed-file inventory from GGUF headers. Flash’s `per_layer_token_embd.weight` lookup table is outside the routed-expert tensor set. File bpw includes all tensors and container overhead.

2.3 Timing protocol and storage counters

Each generation invocation used one fixed technical-explanation prompt, a 4096-token context limit, a request for 256 output tokens, and llama.cpp’s `-st` timing output. The harness saved the reported prompt and generation throughput but not the generated text for these timing runs. It did not set a fixed seed or force greedy decoding. The exact prompt was: *Write a detailed technical explanation of how an operating system page cache interacts with memory-mapped files during sequential and random access patterns.*

The Flash campaign contains an initial three-run sequence labeled as a loaded host and a later three-run sequence fired by a quiet gate after available memory remained at least 35 GiB and one-minute load remained below 3 for 5 minutes. The gpt-oss campaign contains one initial three-run sequence. These are sequential observations, not randomized or interleaved condition samples. The host-state labels describe the campaign; they do not isolate cache size or background load as a cause.

Immediately before process launch and after process exit, the harness sampled the read-sector counter for the physical NVMe device in `/proc/diskstats`. We call the difference a *whole-invocation device-read delta*. It includes every phase of `llama-cli` and reads by other processes on the same device. The generation rate, by contrast, is llama.cpp’s decode timing. These two intervals must not be divided to obtain decode bytes per token, cache misses, or effective decode bandwidth. Memory fields also differ across groups: the initial Flash rows record post-run `Cached` and pre-run `MemAvailable`, while later groups record post-run `MemAvailable`. We therefore do not put them on one cache-size axis.

2.4 Quality checks and alternate scoring

Separate saved evaluations used frozen subsets constructed with dataset seed 42. The surviving files contain the same 25 MATH and 50 GSM8K item IDs for both models. Both campaigns targeted the same 20-item HumanEval+ subset, although the Flash file is incomplete as described below. The GSM8K subset consists of the first 50 entries of a seed-42 sample of 100 test items sorted by source index, not a direct random sample of 50. Servers used an 8192-token context, temperature 1, top- p 0.95, top- k 20, requested reasoning effort `medium`, and concurrency 1. Maximum requested output was 8192 tokens for MATH, 4096 for HumanEval+, and 4096 for GSM8K [20–23]. The harness sent one user message per item and no separate system message. MATH appended “End your response with: Answer: <final answer>” to the problem; GSM8K appended “End your response with: Answer: <number>” to the question. HumanEval+ asked for the complete runnable Python function, including its signature and imports, in one Python code block with nothing afterward, then supplied the task’s function prompt. MATH is reported both under the original harness scorer and a more lenient post-hoc rescore. The original scorer already extracts an answer, normalizes common LaTeX and fraction forms, and applies numeric tolerance. The post-hoc rescore accepts additional presentation variants and simple numeric equivalents. The original score remains primary; the two scores are not merged. One retained MATH row from each model records a timeout error and was counted incorrect. The harness could retry a failed request automatically; the saved rows do not record attempt counts. Task scores describe retained outcomes, not necessarily one generation attempt per item.

The saved Flash HumanEval+ file contains only 17 rows (16 correct), while the aggregate ledger reports 17/20. Because the 3 missing item rows cannot be audited, we omit the aggregate Flash code score from the scientific result and retain the discrepancy in the artifact. Wilson 95% intervals accompany the row-backed proportions [24].

3 Results

3.1 Observed decode throughput

The 67.564 GiB Flash deployment produced 9.4–14.7 decode tokens/s across its six recorded invocations. The 59.034 GiB gpt-oss deployment produced 8.7–9.3 tokens/s across three invocations. Every principal invocation reached a generation timing result. Table 2 gives the individual rates and the separately scoped storage counters. The ranges are observed extrema, not confidence intervals or estimates of run-to-run variability.

Prompt processing was slower than decode in these runs. The initial Flash group reported 4.9, 5, and 4.9 input tokens/s; the later group reported 9.2, 12, and 3.3. The gpt-oss group reported 3.5, 3.3, and 3.4 input tokens/s. Loading time and prompt processing must therefore be measured alongside decode before using these configurations for latency-sensitive work. The single prompt does not support a prediction for longer contexts.

Artifact	Recorded group	n	Decode tok/s	Whole-invocation device reads (GiB)
Flash	initial loaded label	3	9.4 / 10.4 / 10	104.2 / 102.1 / 103.6
Flash	quiet gate	3	14.7 / 14.4 / 13.9	84.3 / 82.1 / 72
gpt-oss	initial loaded label	3	9.3 / 8.7 / 9.2	126.2 / 124.1 / 120.8

Table 2: Sequential generation observations. The read column is the physical device’s counter change over each complete process invocation, not expert traffic or decode-only I/O. The observations are descriptive because the runs were not randomized and host state changed between groups.

The higher Flash rates in the quiet-gated sequence are an observed association with a different host state, not an estimate of a cache-size treatment effect. Likewise, the declining device-read deltas within that sequence do not prove cache warming: the counter includes unrelated reads and its interval differs from the decode timer. The data establish the observed throughput of the two deployments, not why one sequence was faster.

3.2 Stored tensors and the limits of I/O accounting

Flash contains 37.109 GiB of tensors whose standardized names identify routed experts. Its 26.822 GiB `per_layer_token_embd.weight` table is separate and does not match the routed `*_exps` naming rule. The audit does not infer a more specific architectural role from that name alone. The mean stored routed-expert slice is 1.546 MiB per expert-layer for Flash and 12.64 MiB for gpt-oss, an 8.17-fold storage difference.

From those inventories, reading every active routed-expert slice once with no reuse would amount arithmetically to 185.547 GiB per 256 Flash tokens and 455.032 GiB per 256 gpt-oss tokens. These are counterfactual storage sums, not measured I/O. Because the recorded device deltas span additional tensors and process phases, comparing them with the counterfactual does not yield a page-cache hit rate.

3.3 Sampled task performance

Artifact	Task and scoring rule	Correct/ n	Accuracy	Wilson 95% interval
Flash	MATH, original harness	18/25	72%	52.4–85.7%
Flash	MATH, post-hoc lenient	23/25	92%	75–97.8%
Flash	GSM8K	48/50	96%	86.5–98.9%
gpt-oss	MATH, original harness	12/25	48%	30–66.5%
gpt-oss	MATH, post-hoc lenient	20/25	80%	60.9–91.1%
gpt-oss	HumanEval+	18/20	90%	69.9–97.2%
gpt-oss	GSM8K	46/50	92%	81.2–96.8%

Table 3: Row-backed task checks for the deployed configurations. The Flash HumanEval+ aggregate is omitted because 3 item rows are missing. The two MATH rows score the same saved responses under different disclosed rules.

These samples show that the deployed configurations returned correct answers on several tasks while operating through the measured stacks. The MATH result depends materially on the disclosed scoring rule, so the original and post-hoc values remain separate. These samples do not show that mmap preserves quality, because neither artifact was also evaluated in a fully memory-resident configuration and the artifacts are already quantized.

4 Discussion

4.1 What the throughput baseline is useful for

The measured decode rates give users of these exact files a starting point for a local trial. They also specify a baseline that can be rerun under matched conditions when evaluating an expert cache or offloader. A useful comparison would keep the model file, prompt, generation settings, placement, and host state fixed, and report loading, prompt processing, decode, and total response time separately. The slow prompt rates here make that distinction practical: a tolerable stream of output tokens can follow a long wait for the first one.

4.2 What controls performance remains open

The recorded groups show that throughput varied with host state, but their measurement windows cannot assign that variation to cache residency, background load, storage traffic, or another cause. Explaining decode traffic would require phase-scoped and preferably process-attributed I/O, consistent before/after residency metrics, fixed decoding, and randomized repeated host states. Direct router or tensor-access instrumentation would then be needed to distinguish expert popularity, lookup-table traffic, and reuse.

5 Limitations and threats to validity

This is one machine, one NVMe device, one engine commit, two artifacts, and three sequential observations per principal group. The Flash artifact exceeds physical RAM and gpt-oss is smaller; the paper does not cover models several times larger than host memory. Host activity, thermal state, page residency, and invocation order were not randomized or held fixed. The timing harness retained extracted throughput lines rather than full CLI outputs, and it did not set a fixed seed.

Device reads are whole-invocation, physical-device-global counters. They cannot separate engine I/O from unrelated host reads. They also do not distinguish loading, prefill, decode, and teardown. The memory fields differ across groups. No comparison with a purpose-built offloader, a direct-I/O or memory-locking configuration, or a fully memory-resident run of the same artifact was made.

Quality samples are small. MATH conclusions depend strongly on whether the original harness score or more lenient post-hoc rescore is used, so both are reported. Flash HumanEval+ has an unresolved aggregate/item-row mismatch and is omitted. Model and hardware scope is exact: results should not be generalized to other MoEs, quantizations, storage devices, operating systems, or engine revisions without replication.

6 Related work

EdgeMoE, Fast MoE Offloading, HOBBIT, SliceMoE, DyMoE, and PagedWeight represent an engineered lineage of expert placement, caching, prefetching, and mixed precision [8–13]. A proposed llama.cpp 0_DIRECT expert streamer targets pools far beyond RAM and reports improvements over mmap [25], and SSD-LLaMA builds a three-tier SSD, RAM, and VRAM expert hierarchy into llama.cpp to run trillion-parameter MoEs on a consumer PC [26]. Those systems pursue speed, scale, or predictable resource control. The measurements here document an unmodified mmap configuration that could be reused in a future matched comparison; no such comparison is made here.

An opt-in GPU-resident LRU cache for host-offloaded experts is proposed in llama.cpp pull request 27861 [27], still an open draft when checked on September 28, 2026. Its author reports 18.4 versus 24.2 decode tokens/s in a matched cache-disabled/cache-enabled test with a larger Flash quantization on a dual-Xeon, two-RTX-3090 system. A community port of the same cache with an added admission gate reports 5.6 versus 10.0 decode tokens/s for Flash IQ3_XXS on a 12 GB RTX 3060 with 64 GB of RAM, in interleaved arms on one binary [27]. These within-system comparisons test an optimization. Its speedup cannot be transferred to the present laptop, which used another artifact, placement, and engine revision. Testing the cache here would require a new matched experiment.

A later community fork reports 55.54 decode tokens/s without speculation and 61.68 with multi-token prediction for Flash UD-Q3_K_XL on a 62.14 GiB RAM desktop with a 16 GB RTX 5070 Ti [28]. Its September 14 runs use expert caching, lazy loading of the per-layer embedding table, and other engine changes. These are author-reported short-prompt results, not independently reproduced measurements here. Similar RAM capacity does not make the CPU, GPU, quantization, placement, prompt, or software comparable. The present report preserves an older, pinned baseline; it does not recommend that revision over newer runtimes or estimate their speedup on this laptop.

Community reports already establish storage-backed mmap execution [3, 4]; the contribution here is the auditable configuration and measurements for these exact deployments. Recent deployment benchmarks also address adjacent questions: Alfarizy et al. compare a smaller MoE with dense models on consumer and edge devices [29], while a community Flash report sweeps placement and loading modes on one RTX PRO 6000 workstation with 96 GB system RAM [30]. Neither is a matched comparison with the two laptop deployments reported here.

7 Artifact availability and reproducibility

The reproducibility artifact accompanying this report contains checksummed, sanitized timing measurements and item-level quality outcomes, the claims data, environment and artifact manifests, model revisions, and a deterministic analysis-only checker. Without executing a model, the checker verifies the exported files, recomputes reported generation rates and row-backed task scores, and checks the reported artifact byte lengths, effective bpw, and routed-expert totals. It does not include or parse the private working logs or local GGUF headers. Full replication requires downloading the pinned upstream artifacts and running the documented commands on compatible hardware; the weights are linked rather than redistributed. The version-1.0.0 artifact is identified by [doi:10.5281/zenodo.22314465](https://doi.org/10.5281/zenodo.22314465). Its code, sanitized evidence, and replication instructions share the

immutable [versioned GitHub artifact path](#). The DOI identifies the frozen reproducibility bundle; it is not a separate manuscript DOI or a claim of peer review.

8 Conclusion

On a 62 GiB RAM, 12 GB VRAM laptop, unmodified llama.cpp produced 9.4–14.7 decode tokens/s with the 67.564 GiB Qwen3.8-Flash-Next artifact and 8.7–9.3 tokens/s with the 59.034 GiB gpt-oss-120b artifact. The archived invocations and configuration provide a baseline for repeating these deployments and testing changes under matched conditions. Slow prompt processing also shows why decode throughput must be separated from response latency. The observations characterize these files on this machine; their short sequential design does not explain the speed differences or establish quality preservation.

AI assistance. Claude (Anthropic) assisted with evaluation harness development and experiment orchestration. Claude and Codex (OpenAI) assisted with evidence auditing, analysis and figure code, citation checking, and manuscript drafting and revision. Matthew Schwartz directed the research and is responsible for its methods, results, interpretation, claims, citations, rights, and released artifacts.

References

- [1] Linux man-pages project. *mmap(2): Map Files or Devices into Memory*. Linux man-pages project, 2026. URL <https://man7.org/linux/man-pages/man2/mmap.2.html>. Accessed 2026-08-29.
- [2] Linux kernel contributors. *Memory Management Concepts Overview*. The Linux Kernel documentation, 2026. URL <https://docs.kernel.org/admin-guide/mm/concepts.html>. Accessed 2026-08-29.
- [3] qdacsxv. Mmap faster than direct I/O for MoE models. llama.cpp community discussion 18758, 2026. URL <https://github.com/ggml-org/llama.cpp/discussions/18758>. Community development record opened 2026-01-11; accessed 2026-08-29; not peer reviewed.
- [4] kisasexypanter94. RFC+PoC: MoE offload to disk with on-demand paging. llama.cpp community discussion 23324, 2026. URL <https://github.com/ggml-org/llama.cpp/discussions/23324>. Community development record opened 2026-05-19; accessed 2026-08-29; not peer reviewed.
- [5] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziars, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017. URL <https://arxiv.org/abs/1701.06538>.
- [6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2021. URL <https://arxiv.org/abs/2101.03961>. Published in the Journal of Machine Learning Research, 2022.
- [7] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models, 2024. URL <https://arxiv.org/abs/2401.06066>.
- [8] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. EdgeMoE: Empowering sparse large language models on mobile devices, 2023. URL <https://arxiv.org/abs/2308.14352>.
- [9] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading, 2023. URL <https://arxiv.org/abs/2312.17238>. Technical report.
- [10] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. HOBBIT: A mixed precision expert offloading system for fast MoE inference, 2024. URL <https://arxiv.org/abs/2411.01433>.
- [11] Yuseon Choi, Sangjin Kim, Jungjun Oh, Gwangtae Park, Byeongcheol Kim, and Hoi-Jun Yoo. SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference, 2025. URL <https://arxiv.org/abs/2512.12990>. Design Automation Conference 2026.
- [12] Yuegui Huang, Zhiyuan Fang, Weiqi Luo, Ruoyu Wu, Wuhui Chen, and Zibin Zheng. DyMoE: Dynamic expert orchestration with mixed-precision quantization for efficient MoE inference on edge, 2026. URL <https://arxiv.org/abs/2603.19172>.
- [13] Yuchen Yang, Yifan Zhao, Anisha Dasgupta, and Sasa Misailovic. PagedWeight: Efficient MoE LLM serving with dynamic quality-aware weight quantization, 2026. URL <https://arxiv.org/abs/2607.16184>.
- [14] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. Software repository, 2023. URL <https://github.com/ggml-org/llama.cpp>. Qwen3.8 measurements used source revision <https://github.com/ggml-org/llama.cpp/commit/035e22731a7fd70b9854b3a2d64ec68e9b1a45d3>; accessed 2026-08-29.
- [15] danielhanchen and llama.cpp contributors. model: add Qwen3.8-Flash-Next (qwen4exp). llama.cpp pull request 27742, 2026. URL <https://github.com/ggml-org/llama.cpp/pull/27742>. Merged 2026-08-27; merge commit

- 6c84c7d5d8833c6e0df69628f75a0f599797934e; the tested revision 035e22731a7fd70b9854b3a2d64ec68e9b1a45d3 is among the pull request's commits.
- [16] Qwen Team. Qwen3.8-Flash-Next model card. Hugging Face model repository, 2026. URL <https://huggingface.co/Qwen/Qwen3.8-Flash-Next/tree/de4b8e4d43b917e7706784d8bb445c9af86a3540>. Repository revision de4b8e4d43b917e7706784d8bb445c9af86a3540; accessed 2026-08-29.
 - [17] Unsloth. Qwen3.8-Flash-Next GGUF quantizations. Hugging Face model repository, 2026. URL https://huggingface.co/unsloth/Qwen3.8-Flash-Next-GGUF/tree/d3bc75ee6cccf3efc1e228ec00a6cc2cdb1e2249/UD-IQ1_S. Exact deployed tier: UD-IQ1_S; repository revision d3bc75ee6cccf3efc1e228ec00a6cc2cdb1e2249; accessed 2026-08-29.
 - [18] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>. Official model repository: <https://huggingface.co/openai/gpt-oss-120b/tree/b5c939de8f754692c1647ca79fbf85e8c1e70f8a>.
 - [19] ggml-org. gpt-oss-120b GGUF. Hugging Face model repository, 2025. URL <https://huggingface.co/ggml-org/gpt-oss-120b-GGUF/blob/238abdd290bb874b90a5da1b4549881b7d05c091/gpt-oss-120b-MXFP4.gguf>. Exact deployed artifact: gpt-oss-120b-MXFP4.gguf; repository revision 238abdd290bb874b90a5da1b4549881b7d05c091; accessed 2026-08-29.
 - [20] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset, 2021. URL <https://arxiv.org/abs/2103.03874>. NeurIPS 2021.
 - [21] Mark Chen et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
 - [22] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation, 2023. URL <https://arxiv.org/abs/2305.01210>.
 - [23] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
 - [24] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953. URL <https://doi.org/10.1080/01621459.1927.10502953>.
 - [25] freedomljc and llama.cpp contributors. llama: Stream MoE routed experts from disk. llama.cpp pull request 25294, 2026. URL <https://github.com/ggml-org/llama.cpp/pull/25294>. Open development record created 2026-07-04; head revision 1248fd8fa8cfbaece5ea992e4d951c1e18bb9d5 as accessed 2026-08-29; not peer reviewed.
 - [26] Fangzhou Liang, Yibin Shen, Jianmin Hu, Jiayang Xu, Hanchi Gao, Minxian Xu, and Zili Meng. SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC, 2026. URL <https://arxiv.org/abs/2609.18110>. arXiv 2609.18110v1, submitted 2026-09-16.
 - [27] csantiago78 and llama.cpp contributors. llama: GPU-resident LRU cache for host-offloaded MoE expert weights. llama.cpp pull request 27861, 2026. URL <https://github.com/ggml-org/llama.cpp/pull/27861>. Open draft; head bccbacdb8945680f1cfc7e6bfd1e59014705750, checked 2026-09-28; the RTX 3060 port is the comment by pjsjgy of 2026-09-18; not peer reviewed.
 - [28] GenerelSchwerz. Qwen3.8 Flash Next reproduction protocol. Community llama.cpp fork benchmark and reproduction record, 2026. URL <https://github.com/GenerelSchwerz/llama.cpp/wiki/Qwen3.8-Flash-Next-Reproduction/162ffa89354b371b917a450da968cdd70acb4b95>. Runs dated 2026-09-14; wiki revision of 2026-09-21; engine revision 92593380102129b68a5c526d83cb59eb608a0b6d; accessed 2026-09-28; not peer reviewed or independently reproduced here.
 - [29] Alfarizy Alfarizy, Hung Truong Thanh Nguyen, René Richard, Roozbeh Razavi-Far, and Hung Cao. Does mixture-of-experts actually help inference on consumer and edge hardware? an empirical study, 2026. URL <https://arxiv.org/abs/2606.21428v3>. Version 3, July 9, 2026; checked 2026-09-05.
 - [30] lukaLLM. Qwen3.8-Flash-Next on one RTX PRO 6000. Community benchmark repository, 2026. URL <https://github.com/lukaLLM/Qwen3.8-Flash-Next-VRAM-Benchmark/tree/518a52dafcfc707ca2f7d5349635350d0b2fe831>. Repository revision 518a52dafcfc707ca2f7d5349635350d0b2fe831; checked 2026-09-05; not peer reviewed.